

Build Phase 1 — Core App (1.5 hours)

Goal

By the end of this phase, you should have a **minimal working app**: a frontend that sends data to an API, and the API returns a response that's displayed to the user. Keep the scope intentionally small so you have plenty of time for Playwright in Phases 2 and 3.

Step 1: Plan and Generate Your Prompts (10-15 min)

Follow the **Copilot Workflow Guide** (`copilot-workflow-guide.md`) to:

1. Use **Plan agent** to plan your Phase 1 approach — describe your use case and stack
2. Switch to **Agent Mode** and ask Copilot to break the plan into step-by-step prompts
3. Save the prompts to a `phase1-prompts.md` file in your project

Once you have your prompts saved, work through the remaining steps below — or follow your own generated prompts if they feel more natural. Either way, use your prompts one at a time and test after each step.

Step 2: Set Up the Project (10-15 min)

Create your project folder and initialize your stack.

Examples by stack:

- **JavaScript:** `npx create-react-app frontend` + `npm init` in an api/ folder, install Express`
- **Python:** Create a `frontend/` folder with HTML files + `pip install flask` or `pip install fastapi uvicorn``
- **Java:** Use Spring Initializr or `mvn archetype:generate` for a Spring Boot project`
- **.NET:** `dotnet new webapi` + `dotnet new blazorserver` or Razor Pages`

***Copilot tip:** Switch to **Agent Mode** and ask it to set up the project structure for you. e.g., `Set up a new Express API project in the api/ folder with a POST /validate endpoint that returns JSON.`*

Step 3: Build the Frontend (20-25 min)

Create a simple form that collects the fields for your document type.

What to build:

- A form with input fields matching your use case (e.g., vendor name, amount, date for an invoice)
- A submit button
- An area to display results

Keep it simple. A plain HTML form with no styling is perfectly fine. Don't spend time on CSS yet — your biggest investment after this phase is Playwright.

***Copilot tip:** Describe what you want in a comment or chat: `Create a form with fields for vendor name, invoice amount, invoice date, and PO number. Add a submit button and a results div.`*

Step 4: Build the API (20-25 min)

Create an endpoint that receives the form data and returns a response.

What to build:

- A POST endpoint (e.g., `/validate`` or `/api/validate``)
- Accept JSON input from the frontend
- For now, just echo back the data or return a simple `{ "valid": true }` response
- Make sure CORS is configured if frontend and API run on different ports

Copilot tip: Create a POST `/validate` endpoint that accepts JSON with fields `vendorName`, `amount`, `date`, and `poNumber`. Return a JSON response with `valid: true` and a message.

Step 5: Connect Frontend to API (15-20 min)

Wire up the form submission to call your API and display the result.

What to do:

- On form submit, send a `fetch()` or `axios`` POST request to your API
- Parse the JSON response
- Display the result in the results area

Test it: Fill in the form, submit, and verify the API response appears on screen.

Copilot tip: If you get CORS errors or connection issues, paste the error into Copilot Chat and ask for help.

Step 6: Add Basic Validation Logic (20-25 min)

Now replace the placeholder response with actual validation.

Start with 2-3 simple rules:

- Is a required field missing or empty?
- Is a number positive?
- Is a date in a valid range?

Return structured results:

```
{ "valid": false, "errors": [ "Invoice amount must be positive", "PO number is required" ] }
```

Display each error on the frontend so the user can see what failed.

Copilot tip: Add validation logic: check that `vendorName` is not empty, `amount` is a positive number, `date` is not in the future, and `poNumber` matches the format `PO-XXXX`.

Checkpoint

By now you should have:

- [] A frontend with a form
- [] An API endpoint that receives and validates data
- [] Validation results displayed back to the user

- [] At least 2-3 validation rules working

****If you're ahead of schedule:**** Start thinking about the three Playwright scenarios you want to cover in Phases 2 and 3: a happy path, a negative path, and one richer scenario.

****If you're behind:**** Focus on getting the API to return *any* response and displaying it. Skip fancy validation for now — the goal is a minimal app that is ready to test with Playwright next.